

Technical Specification: GymCore – Fitness SaaS Platform

Document: Initial Product Requirements and Architecture Document (PRD/Arch)

Application Type: Containerized End-to-End (E2E) Application

Architectural Pattern: Modular Monolith API + Single Page Application (SPA)

1. System Objectives and Scope

GymCore is a comprehensive management system for boutique fitness clubs. The software integrates multi-tier subscription sales, strict facility access control, and a highly concurrent booking engine for group classes and personal training sessions. The project strongly emphasizes performance, transactional safety, and architectural scalability.

2. Role-Based Access Control (RBAC)

System resources are secured using JSON Web Tokens (JWT). The application implements three distinct permission levels:

- **User (Client):** Browses the service catalog, purchases and manages subscriptions, books group classes, schedules personal training sessions, and views transaction history.
- **Trainer (Employee):** Defines availability windows (time slots) for 1-on-1 sessions and accesses attendee lists for assigned group classes.
- **Administrator (Manager):** Has full CRUD (Create, Read, Update, Delete) access to system resources. Configures facilities, equipment, pricing tiers, user accounts, and reviews financial analytics.

3. Core Business Modules

3.1. Infrastructure and Catalog Module

- **Room Management:** Defines physical zones within the facility (e.g., Free Weights Zone, SPA). Each zone requires a specific minimum subscription tier for entry.
- **Service Catalog:** A public-facing registry of available group classes and instructor profiles, including their specializations.

3.2. Subscriptions and Access Control Module

- **Subscription Lifecycle:** Implements a state machine to manage membership statuses (Active, Expiring, Expired, Cancelled).
- **Access Control Endpoint:** A dedicated endpoint simulating a physical entry scanner, verifying user permissions for specific facility zones in real-time.
- **Dynamic Pricing Engine:** Calculates costs for supplementary services (e.g., personal training) on the fly. Higher-tier memberships automatically apply percentage-based discounts before transaction finalization.

3.3. Booking Engine Module

- **Group Classes:** Handles class reservations with strict attendee limits, engineered to withstand high-concurrency environments.
- **Personal Training:** A time-allocation system based on individual trainer availability calendars.
- **Waitlist Processing:** An automated background process that promotes users from the waitlist to the main attendee list upon cancellations.

4. Technology Stack

4.1. Server-Side (Backend)

- **Environment:** C# 13, .NET 9, ASP.NET Core Web API.
- **Architecture:** Clean Architecture with CQRS pattern implementation using the MediatR library.
- **Database:** PostgreSQL.
- **ORM:** Entity Framework Core (Code-First approach).
- **Validation:** FluentValidation for strict input data verification.
- **Security:** Integrated Identity / custom BCrypt implementation for password hashing and JWT-based authentication.

4.2. Client-Side (Frontend)

- **Environment:** React 18, Vite (build tool and development server), Node.js (local runtime environment).
- **Language:** TypeScript (strict typing reflecting API contracts).
- **Asynchronous State Management:** @tanstack/react-query for automated request caching and loading state management.
- **User Interface:** Tailwind CSS integrated with modern component libraries like shadcn/ui or Radix UI.
- **Form Handling:** react-hook-form paired with the zod library for robust schema validation.

4.3. Infrastructure and Quality Assurance

- **Containerization:** Environment fully managed via docker-compose.yml (isolated containers for the database, API, and an Nginx web server serving the frontend build).
- **Configuration Management:** Environment variables strictly separated from source code (.env files).
- **Testing & Error Handling:** Unit tests for the domain layer (xUnit, Moq) and integration tests for critical endpoints. A Global Exception Handler transforms errors into the standardized ProblemDetails format.

5. Architectural Challenges

- **Concurrency Management:** Preventing overbooking by implementing Optimistic Locking (e.g., utilizing a RowVersion column in EF Core).
- **Background Processing:** Leveraging the IHostedService interface to execute thread-safe background tasks (e.g., waitlist resolution).
- **Advanced SQL Integration:** Integrating EF Core with SQL Views to optimally generate administrative reports without overloading the application logic.

6. Project Implementation Phases

1. **Infrastructure:** Initialize the repository, configure the Docker environment (PostgreSQL), and scaffold the base .NET and Vite/React projects.
2. **Domain and Authorization:** Design entity models, configure relational mappings (Fluent API), and implement the authentication system and RBAC.
3. **Business Logic (API):** Implement CQRS, transaction handling, input validation, and the booking engine with concurrency locks.
4. **Client Interface:** Build the UI layout, implement forms, and consume API endpoints using React Query.
5. **Refinement and Optimization:** Deploy background tasks, integrate SQL procedures, execute test coverage, and finalize documentation (README, ERD).